

AutoChip Tutorial Assignment -- Report

Course: LLM4ChipDesign | Student: Anuj Apte

Overview

This report summarizes running AutoChip's generate → compile (iverilog) → simulate (vvp) → feedback → regenerate loop on all eight tutorial examples and the accumulator-based 8-bit CPU stretch example. The accompanying notebook (AutoChip_hw.ipynb) executes the real AutoChip control-flow code (Conversation bookkeeping, LLMResponse ranking, iterative feedback messages) from scratch against real iverilog/vvp output for every example below, with no fabricated results. One adaptation was required: the provided testbenches for these examples are self-checking (they print pass/fail text directly) rather than following the VerilogEval “Mismatches: X in Y samples” convention AutoChip's stock rank function expects, so the pass/fail detection in calculate_rank was adapted accordingly while the rest of the loop (conversation growth, iteration cap, compile/simulate feedback messages) is unchanged from AutoChip.

Each example's Verilog was produced by the assignment author (using Claude) acting as the AutoChip generation step -- reading the same conversation state AutoChip sends an LLM API and returning a Verilog module -- with real iverilog/vvp tool feedback driving each subsequent attempt exactly as AutoChip's automatic loop would with a live API key.

Summary table

Example	Iterations	Final status
binary_to_bcd	1	PASS
sequence_detector	2	PASS
shift_register	2	FAIL (testbench issue -- see below)
abro_state_machine	1	PASS
dice_roller	1	PASS
lfsr	1	PASS
sequence_generator	2	PASS
traffic_light	1	PASS
cpu_8bit (Part II)	1	PASS

Feedback-driven change highlighted for this assignment: sequence_detector and sequence_generator (see their sections below) both required a real RTL restructuring in response to iverilog/vvp simulation feedback, not just a cosmetic fix.

Example 1 -- binary_to_bcd (combinational converter)

Specification given to AutoChip

I am trying to create a Verilog model for a binary to binary-coded-decimal converter. It must meet the follow

■- Inputs:

■■- Binary input (5-bits)

■- Outputs:

■■- BCD (8-bits: 4-bits for the 10's place and 4-bits for the 1's place)

How would I write a design that meets these specifications?

AutoChip configuration / parameters

```
model_family = "manual-AutoChip-loop" (assignment author acting as the LLM step)
module       = "binary_to_bcd_converter"
testbench    = "binary_to_bcd_tb.v" (top module: tb_binary_to_bcd_converter)
iterations   = 1 (max_iterations=0)
num_candidates = 1
```

Generation trajectory

Iteration 0: PASS

Verification

```
iverilog -Wall -Winfloop -Wno-timescale -g2012 -s tb_binary_to_bcd_converter -o binary_to_bcd_converter.vvp b
vvp -n binary_to_bcd_converter.vvp
```

Simulator output:

Testing Binary-to-BCD Converter...

VCD info: dumpfile my_design.vcd opened for output.

All test cases passed!

/home/claude/HW_AutoChip/examples/binary_to_bcd/binary_to_bcd_tb.v:38: \$finish called at 320000 (1ps)

Discussion

Combinational converter; iverilog compiled cleanly and the self-checking testbench (32 exhaustive cases) passed on the first generated attempt.

Final module

```
module binary_to_bcd_converter (
    input  [4:0] binary_input,
    output reg [7:0] bcd_output
);
    always @(*) begin
        bcd_output[7:4] = binary_input / 10;
        bcd_output[3:0] = binary_input % 10;
    end
endmodule
```

Example 2 -- sequence_detector (FSM sequence matching)

Specification given to AutoChip

I am trying to create a Verilog model for a sequence detector. It must meet the following specifications:

- Inputs:
- Clock
- Active-low reset
- Data (3 bits)
- Outputs:
- Sequence found

While enabled, it should detect the following sequence of binary input values:

- 0b001
- 0b101
- 0b110
- 0b000
- 0b110
- 0b110
- 0b011
- 0b101

How would I write a design that meets these specifications?

AutoChip configuration / parameters

```
model_family = "manual-AutoChip-loop" (assignment author acting as the LLM step)
module       = "sequence_detector"
testbench    = "sequence_detector_tb.v" (top module: tb_sequence_detector)
iterations   = 2 (max_iterations=1)
num_candidates = 1
```

Generation trajectory

Iteration 0: SIMULATION MISMATCH

Tool feedback fed back into the next attempt:

```
Error: Cycle 8, Expected: 1, Got: 0
/home/claude/HW_AutoChip/examples/sequence_detector/sequence_detector_tb.v:39: $finish called at 85000 (1ps)
```

Iteration 1: PASS

Verification

```
iverilog -Wall -Winfloop -Wno-timescale -g2012 -s tb_sequence_detector -o sequence_detector.vvp sequence_detector.vvp -n sequence_detector.vvp
```

Simulator output:

```
All test cases passed.
/home/claude/HW_AutoChip/examples/sequence_detector/sequence_detector_tb.v:73: $finish called at 125000 (1ps)
```

Discussion

First attempt registered `sequence\_found` from a fully-registered 8-symbol shift history; iverilog compiled it cleanly but simulation failed at the final symbol because the testbench samples the output in the same simulation step as the clock edge, before the DUT's nonblocking updates settle -- so a purely registered comparison is always read one cycle stale. Iteration 2 fixed this by keeping only the previous 7 symbols in a register and comparing them combinationaly against the live `data` input plus the target pattern, which removed the extra register stage feeding the checked output. This is the example highlighted for the report as showing a

meaningful, feedback-driven RTL change.

Final module

```
module sequence_detector (  
    input clk,  
    input reset_n,  
    input [2:0] data,  
    output sequence_found  
);  
    // History holds the previous 7 symbols; the testbench samples  
    // sequence_found combinationally right after asserting the new  
    // symbol and before the next clock edge's registers have settled,  
    // so the comparison must be combinational (current data + the  
    // already-registered previous 7 symbols) rather than fully  
    // registered, to avoid a one-cycle lag against the checker.  
    reg [20:0] history;  
    localparam [23:0] TARGET = {3'b001,3'b101,3'b110,3'b000,3'b110,3'b110,3'b011,3'b101};  
  
    always @(posedge clk or negedge reset_n) begin  
        if (!reset_n)  
            history <= 21'b0;  
        else  
            history <= {history[17:0], data};  
    end  
  
    assign sequence_found = ({history, data} == TARGET);  
endmodule
```

Example 3 -- shift_register (sequential shift register)

Specification given to AutoChip

I am trying to create a Verilog model for a shift register. It must meet the following specifications:

- Inputs:
- Clock
- Active-low reset
- Data (1 bit)
- Shift enable
- Outputs:
- Data (8 bits)

How would I write a design that meets these specifications?

AutoChip configuration / parameters

```
model_family = "manual-AutoChip-loop" (assignment author acting as the LLM step)
module       = "shift_register"
testbench    = "shift_register_tb.v" (top module: tb_shift_register)
iterations   = 2 (max_iterations=1)
num_candidates = 1
```

Generation trajectory

Iteration 0: SIMULATION MISMATCH

Tool feedback fed back into the next attempt:

```
Error: Test case 0 failed. Expected: 00001010, Got: xxxxxxxx
/home/claude/HW_AutoChip/examples/shift_register/shift_register_tb.v:47: $finish called at 10000 (1ps)
```

Iteration 1: SIMULATION MISMATCH

Tool feedback fed back into the next attempt:

```
Error: Test case 0 failed. Expected: 00001010, Got: 00000000
/home/claude/HW_AutoChip/examples/shift_register/shift_register_tb.v:47: $finish called at 10000 (1ps)
```

Verification

```
iverilog -Wall -Winfloop -Wno-timescale -g2012 -s tb_shift_register -o shift_register.vvp shift_register.sv s
vvp -n shift_register.vvp
```

Simulator output:

```
Error: Test case 0 failed. Expected: 00001010, Got: 00000000
/home/claude/HW_AutoChip/examples/shift_register/shift_register_tb.v:47: $finish called at 10000 (1ps)
```

Discussion

First attempt left data_out undefined (X) because the testbench never actually pulses reset_n low before its first check; iteration 2 added a simulation-time initial value. The testbench still fails: its expected-output constant (test_case_data_out) does not depend on data_in/shift_enable at all -- every checked value equals 10 right-shifted by the loop index, which we verified directly against Icarus rather than against our own arithmetic. No design that actually implements the specified shift register (or any alternative shift direction / enable interpretation we tried) can reproduce that input-independent sequence, so this looks like a bug in the provided testbench's reference data rather than in the DUT. This is documented as the remaining, unresolved issue for this example.

Final module

```
module shift_register (
    input clk,
    input reset_n,
```

```

    input data_in,
    input shift_enable,
    output reg [7:0] data_out
);
    // Give data_out a defined value at t=0 in addition to the
    // asynchronous active-low reset, since the testbench never
    // pulses reset_n low before its first check.
    initial data_out = 8'b0;

    always @(posedge clk or negedge reset_n) begin
        if (!reset_n)
            data_out <= 8'b0;
        else if (shift_enable)
            data_out <= {data_out[6:0], data_in};
    end
endmodule

```

Example 4 -- abro_state_machine (ABRO handshake style FSM)

Specification given to AutoChip

I am trying to create a Verilog model for an ABRO state machine. It must meet the following specifications:

- Inputs:
 - Clock
 - Active-low reset
 - A
 - B
- Outputs:
 - O
 - State

Other than the main output from ABRO machine, it should output the current state of the machine for use in verification.

The states for this state machine should be one-hot encoded.

How would I write a design that meets these specifications?

AutoChip configuration / parameters

```
model_family = "manual-AutoChip-loop" (assignment author acting as the LLM step)
module       = "abro_state_machine"
testbench    = "abro_state_machine_tb.v" (top module: tb_abro_state_machine)
iterations   = 1 (max_iterations=0)
num_candidates = 1
```

Generation trajectory

Iteration 0: PASS

Verification

```
iverilog -Wall -Winfloop -Wno-timescale -g2012 -s tb_abro_state_machine -o abro_state_machine.vvp abro_state_
vvp -n abro_state_machine.vvp
```

Simulator output:

VCD info: dumpfile my_design.vcd opened for output.

All test cases passed.

/home/claude/HW_AutoChip/examples/abro_state_machine/abro_state_machine_tb.v:90: \$finish called at 165000 (1p

Discussion

One-hot ABRO FSM with an explicit hold-state for the A=B=0 case; compiled and passed all 16 testbench cases on the first attempt.

Final module

```
module abro_state_machine (
    input clk,
    input rst_n,
    input A,
    input B,
    output reg O,
    output reg [3:0] State
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            State <= 4'b0001;
            O <= 1'b0;
        end else begin
```

```
    0 <= (A && B) ? 1'b1 : 1'b0;
    if (A && !B)
        State <= 4'b0010;
    else if (!A && B)
        State <= 4'b0001;
    else if (A && B)
        State <= 4'b0100;
    // else A==0 && B==0: hold current State
end
end
endmodule
```

Example 5 -- dice_roller (randomized behavior)

Specification given to AutoChip

I am trying to create a Verilog model for a simulated dice roller. It must meet the following specifications:

- Inputs:
 - Clock
 - Active-low reset
 - Die select (2-bits)
 - Roll
- Outputs:
 - Rolled number (up to 8-bits)

The design should simulate rolling either a 4-sided, 6-sided, 8-sided, or 20-sided die, based on the input die select.

How would I write a design that meets these specifications?

AutoChip configuration / parameters

```
model_family = "manual-AutoChip-loop" (assignment author acting as the LLM step)
module        = "dice_roller"
testbench     = "dice_roller_tb.v" (top module: tb_dice_roller)
iterations    = 1 (max_iterations=0)
num_candidates = 1
```

Generation trajectory

Iteration 0: PASS

Verification

```
iverilog -Wall -Winfloop -Wno-timescale -g2012 -s tb_dice_roller -o dice_roller.vvp dice_roller.sv dice_roller_tb.sv
vvp -n dice_roller.vvp
```

Simulator output:

VCD info: dumpfile my_design.vcd opened for output.

Results for die_select 00:

Rolled	1:	245 times
Rolled	2:	253 times
Rolled	3:	251 times
Rolled	4:	251 times

Results for die_select 01:

Rolled	1:	164 times
Rolled	2:	170 times
Rolled	3:	165 times
Rolled	4:	170 times
Rolled	5:	164 times
Rolled	6:	167 times

Results for die_select 10:

Rolled	1:	121 times
Rolled	2:	128 times
Rolled	3:	125 times
Rolled	4:	125 times
Rolled	5:	

... (truncated)

Discussion

Free-running 8-bit Fibonacci LFSR reduced modulo the selected die's side count; passed the 4x1000-roll range/statistics check on the first attempt.

Final module

```
module dice_roller (
```

```

    input clk,
    input rst_n,
    input [1:0] die_select,
    input roll,
    output reg [7:0] rolled_number
);
    reg [7:0] lfsr;
    wire feedback = lfsr[7] ^ lfsr[5] ^ lfsr[4] ^ lfsr[3];

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            lfsr <= 8'hA5;
        else
            lfsr <= {lfsr[6:0], feedback};
    end

    reg [4:0] sides;
    always @(*) begin
        case (die_select)
            2'b00: sides = 5'd4;
            2'b01: sides = 5'd6;
            2'b10: sides = 5'd8;
            default: sides = 5'd20;
        endcase
    end

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            rolled_number <= 8'd1;
        else if (roll)
            rolled_number <= (lfsr % sides) + 1'b1;
    end
endmodule

```

Example 6 -- lfsr (linear feedback shift register)

Specification given to AutoChip

I am trying to create a Verilog model for an LFSR. It must meet the following specifications:

- Inputs:
- Clock
- Active-low reset
- Outputs:
- Data (8-bits)

The initial state should be 10001010, and the taps should be at locations 1, 4, 6, and 7.

How would I write a design that meets these specifications?

AutoChip configuration / parameters

```
model_family = "manual-AutoChip-loop" (assignment author acting as the LLM step)
module       = "lfsr"
testbench    = "lfsr_tb.v" (top module: tb_lfsr)
iterations   = 1 (max_iterations=0)
num_candidates = 1
```

Generation trajectory

Iteration 0: PASS

Verification

```
iverilog -Wall -Winfloop -Wno-timescale -g2012 -s tb_lfsr -o lfsr.vvp lfsr.sv lfsr_tb.v
vvp -n lfsr.vvp
```

Simulator output:

```
Simulation successful. All test cases passed.
/home/claude/HW_AutoChip/examples/lfsr/lfsr_tb.v:67: $finish called at 2565000 (1ps)
```

Discussion

Direct translation of the tap/seed spec; matched the testbench's bit-exact 256-cycle reference sequence on the first attempt.

Final module

```
module lfsr (
    input  clk,
    input  reset_n,
    output reg [7:0] data
);
    wire feedback = data[0] ^ data[3] ^ data[5] ^ data[6];

    always @(posedge clk or negedge reset_n) begin
        if (!reset_n)
            data <= 8'b10001010;
        else
            data <= {data[6:0], feedback};
    end
endmodule
```

Example 7 -- sequence_generator (pattern generator)

Specification given to AutoChip

I am trying to create a Verilog model for a sequence generator. It must meet the following specifications:

- Inputs:
- Clock
- Active-low reset
- Enable
- Outputs:
- Data (8 bits)

While enabled, it should generate an output sequence of the following hexadecimal values and then repeat:

- 0xAF
- 0xBC
- 0xE2
- 0x78
- 0xFF
- 0xE2
- 0x0B
- 0x8D

How would I write a design that meets these specifications?

AutoChip configuration / parameters

```
model_family = "manual-AutoChip-loop" (assignment author acting as the LLM step)
module       = "sequence_generator"
testbench    = "sequence_generator_tb.v" (top module: tb_sequence_generator)
iterations   = 2 (max_iterations=1)
num_candidates = 1
```

Generation trajectory

Iteration 0: SIMULATION MISMATCH

Tool feedback fed back into the next attempt:

```
Error: Mismatch at position 1. Expected: bc, Got: af
/home/claude/HW_AutoChip/examples/sequence_generator/sequence_generator_tb.v:58: $finish called at 45000 (1ps)
```

Iteration 1: PASS

Verification

```
iverilog -Wall -Winfloop -Wno-timescale -g2012 -s tb_sequence_generator -o sequence_generator.vvp sequence_ge
vvp -n sequence_generator.vvp
```

Simulator output:

```
Test case passed: sequence generation
Test case passed: sequence repetition
Test case passed: disable sequence generator
All test cases passed
/home/claude/HW_AutoChip/examples/sequence_generator/sequence_generator_tb.v:83: $finish called at 195000 (1p
```

Discussion

First attempt registered `data` directly (index then output both delayed by clocked registers); the testbench again samples immediately after the posedge, before the DUT's own nonblocking updates for that edge settle, so it saw the previous symbol. Iteration 2 made `data` a continuous read of `seq[idx]`, with only `idx` registered, which resolved the same class of race seen in sequence_detector and passed generation, repetition, and disable checks.

Final module

```
module sequence_generator (  
    input clk,  
    input reset_n,  
    input enable,  
    output [7:0] data  
);  
    // The testbench samples `data` immediately after each posedge,  
    // before this edge's own nonblocking updates have settled, so  
    // the output must be combinational off of a register that was  
    // already settled as of the *previous* edge (same pattern as  
    // sequence_detector), rather than itself carrying a one-cycle  
    // register delay.  
    reg [2:0] idx;  
    reg [7:0] seq [0:7];  
  
    initial begin  
        seq[0] = 8'hAF;  
        seq[1] = 8'hBC;  
        seq[2] = 8'hE2;  
        seq[3] = 8'h78;  
        seq[4] = 8'hFF;  
        seq[5] = 8'hE2;  
        seq[6] = 8'h0B;  
        seq[7] = 8'h8D;  
    end  
  
    assign data = seq[idx];  
  
    always @(posedge clk or negedge reset_n) begin  
        if (!reset_n)  
            idx <= 3'd0;  
        else if (enable)  
            idx <= idx + 1'b1;  
    end  
endmodule
```

Example 8 -- traffic_light (traffic light FSM)

Specification given to AutoChip

I am trying to create a Verilog model for a traffic light state machine. It must meet the following specifications:

- Inputs:
 - Clock
 - Active-low reset
 - Enable
- Outputs:
 - Red
 - Yellow
 - Green

The state machine should reset to a red light, change from red to green after 32 clock cycles, change from green to yellow after 20 clock cycles, and change from yellow to red after 7 clock cycles.

How would I write a design that meets these specifications?

AutoChip configuration / parameters

```
model_family = "manual-AutoChip-loop" (assignment author acting as the LLM step)
module       = "traffic_light_fsm"
testbench    = "traffic_light_tb.v" (top module: tb_traffic_light_fsm)
iterations   = 1 (max_iterations=0)
num_candidates = 1
```

Generation trajectory

Iteration 0: PASS

Verification

```
iverilog -Wall -Winfloop -Wno-timescale -g2012 -s tb_traffic_light_fsm -o traffic_light_fsm.vvp traffic_light_fsm.vvp
vvp -n traffic_light_fsm.vvp
```

Simulator output:

```
Test case 1 passed - FSM transitioned to RED after 0 clock cycles.
Test case 2 passed - FSM transitioned to GREEN after 32 clock cycles.
Test case 3 passed - FSM transitioned to YELLOW after 20 clock cycles.
Test case 4 passed - FSM transitioned to RED after 7 clock cycles.
All test cases passed!
/home/claude/HW_AutoChip/examples/traffic_light/traffic_light_tb.v:79: $finish called at 610 (1ps)
```

Discussion

3-state counter-based FSM (RED/GREEN/YELLOW) with the specified 32/20/7 cycle counts; passed all 4 testbench checks on the first attempt.

Final module

```
module traffic_light_fsm (
    input clk,
    input reset_n,
    input enable,
    output reg red,
    output reg yellow,
    output reg green
);
    localparam S_RED = 2'b00, S_GREEN = 2'b01, S_YELLOW = 2'b10;
    reg [1:0] state;
    reg [5:0] count;

    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) begin
```

```

        state <= S_RED;
        count <= 6'd0;
    end else if (enable) begin
        case (state)
            S_RED:    if (count == 6'd31) begin state <= S_GREEN;  count <= 0; end
                       else count <= count + 1'b1;
            S_GREEN:  if (count == 6'd19) begin state <= S_YELLOW; count <= 0; end
                       else count <= count + 1'b1;
            S_YELLOW: if (count == 6'd6)  begin state <= S_RED;    count <= 0; end
                       else count <= count + 1'b1;
            default:  state <= S_RED;
        endcase
    end
end

always @(*) begin
    red = 1'b0; yellow = 1'b0; green = 1'b0;
    case (state)
        S_RED:    red    = 1'b1;
        S_GREEN:  green  = 1'b1;
        S_YELLOW: yellow = 1'b1;
    endcase
end
endmodule

```

Part II -- 8-bit CPU example

Starting co-design prompt (Fig. 10 of the assignment PDF)

Let us make a brand new microprocessor design together. We're severely constrained on space and I/O. We have to fit in 1000 standard cells of an ASIC, so I think we will need to restrict ourselves to an accumulator based 8-bit architecture with no multi-byte instructions. Given this, how do you think we should begin?

Major trajectory steps (architecture co-design)

- 1 Instruction width: "no multi-byte instructions" forces exactly one byte per instruction, so opcode and operand share 8 bits.
- 2 Opcode/operand split: a 3-bit opcode (8 opcodes) leaves 5 operand bits (32 addressable words) -- enough for a small program plus data footprint within a ~1000-cell budget.
- 3 Instruction set: the minimal accumulator set needed for real programs -- NOP, LDA, STA, ADD, SUB, JMP, JZ, HALT.
- 4 Datapath: Harvard split (separate 32x8 program ROM addressed by a 5-bit PC, and 32x8 data RAM addressed by the operand field) rather than a shared bus, to keep control logic single-cycle and small.
- 5 Control: one always block, single-cycle fetch+execute per instruction (no pipeline stages), again to minimize cell count.
- 6 RTL + verification: implemented the above and verified against a testbench that loads a program summing $5+4+3+2+1$ via a decrement loop and checks the final accumulator/memory state and halt behavior.

Tool feedback obtained

The RTL compiled and passed its testbench on the first attempt (no compile or simulation errors were produced), so the iteration effort for this example went into the architecture decisions above rather than into repairing failed RTL:

```
iverilog -Wall -Winfloop -Wno-timescale -g2012 -s tb_cpu_8bit \  
-o cpu_8bit.vvp cpu_8bit.sv cpu_8bit_tb.v  
vvp -n cpu_8bit.vvp  
  
CPU halted after 45 cycles. PC=10 ACC=15  
Result check passed: data_mem[3] = 15  
All CPU test cases passed!  
cpu_8bit_tb.v:73: $finish called at 460000 (1ps)
```

Discussion

How AutoChip-style decomposition mapped onto this design: The ISA, datapath and control questions were resolved in a fixed dependency order (word width -> opcode/operand split -> instruction set -> memory organization -> control style), a much larger decomposition than any of the 8 tutorial examples need.

Does feedback that works for small examples still work here?: Yes for RTL-level bugs -- a wrong case branch or off-by-one would show up in iverilog/vvp output exactly as in the smaller examples -- but it cannot catch an architectural mistake such as too few operand bits to address the needed memory; that has to be caught by reasoning about the stated constraint before RTL is written, not by tool feedback afterward.

Hardest part of the CPU design for the workflow: The constraint-driven ISA design step. Choosing the opcode/operand split and instruction set that fit "1000 standard cells" and "no multi-byte instructions" is a judgment call with no compiler feedback signal at all -- iverilog has no opinion on whether 8 opcodes is the right number, only on whether the Verilog already written for the ISA already chosen compiles and simulates correctly.

Remaining limitations: No interrupts or stack (so no subroutine calls), fixed 32-word program/data memories, undefined behavior on PC overflow past address 31, and no realistic timing/area analysis against the actual 1000-cell target -- only cycle-accurate functional verification was performed here.

Final CPU module

```
// 8-bit accumulator-based microprocessor.
// Co-design constraints from the starting prompt: severely limited
// space/IO (~1000 standard cells), accumulator architecture, no
// multi-byte instructions -> every instruction is exactly one byte.
//
// Instruction format: [7:5] = opcode (3 bits), [4:0] = operand/address (5 bits)
// 000 NOP
// 001 LDA addr  ACC <= DMEM[addr]
// 010 STA addr  DMEM[addr] <= ACC
// 011 ADD addr  ACC <= ACC + DMEM[addr]
// 100 SUB addr  ACC <= ACC - DMEM[addr]
// 101 JMP addr  PC  <= addr
// 110 JZ  addr  if (ACC==0) PC <= addr; else PC <= PC + 1
// 111 HALT      halted <= 1
//
// Harvard split: 32x8 program ROM addressed by 5-bit PC, 32x8 data RAM
// addressed by the 5-bit operand. Single-cycle fetch+execute per instruction.

module cpu_8bit (
    input wire      clk,
    input wire      reset_n,
    output reg [7:0] acc,
    output reg [4:0] pc,
    output reg      halted
);
    localparam OP_NOP  = 3'b000;
    localparam OP_LDA  = 3'b001;
    localparam OP_STA  = 3'b010;
    localparam OP_ADD  = 3'b011;
    localparam OP_SUB  = 3'b100;
    localparam OP_JMP  = 3'b101;
    localparam OP_JZ   = 3'b110;
    localparam OP_HALT = 3'b111;

    reg [7:0] prog_mem [0:31];
    reg [7:0] data_mem [0:31];

    initial $readmemh("program.hex", prog_mem);

    wire [7:0] instr  = prog_mem[pc];
    wire [2:0] opcode = instr[7:5];
    wire [4:0] operand = instr[4:0];

    always @(posedge clk) begin
        if (!reset_n) begin
            acc <= 8'd0;
            pc  <= 5'd0;
            halted <= 1'b0;
        end else if (!halted) begin
            case (opcode)
                OP_NOP: pc <= pc + 5'd1;
                OP_LDA: begin acc <= data_mem[operand]; pc <= pc + 5'd1; end
                OP_STA: begin data_mem[operand] <= acc; pc <= pc + 5'd1; end
                OP_ADD: begin acc <= acc + data_mem[operand]; pc <= pc + 5'd1; end
                OP_SUB: begin acc <= acc - data_mem[operand]; pc <= pc + 5'd1; end
                OP_JMP: pc <= operand;
                OP_JZ:  pc <= (acc == 8'd0) ? operand : pc + 5'd1;
                OP_HALT: halted <= 1'b1;
            endcase
        end
    end
endmodule
```

```
        default: pc <= pc + 5'd1;
    endcase
end
endmodule
```